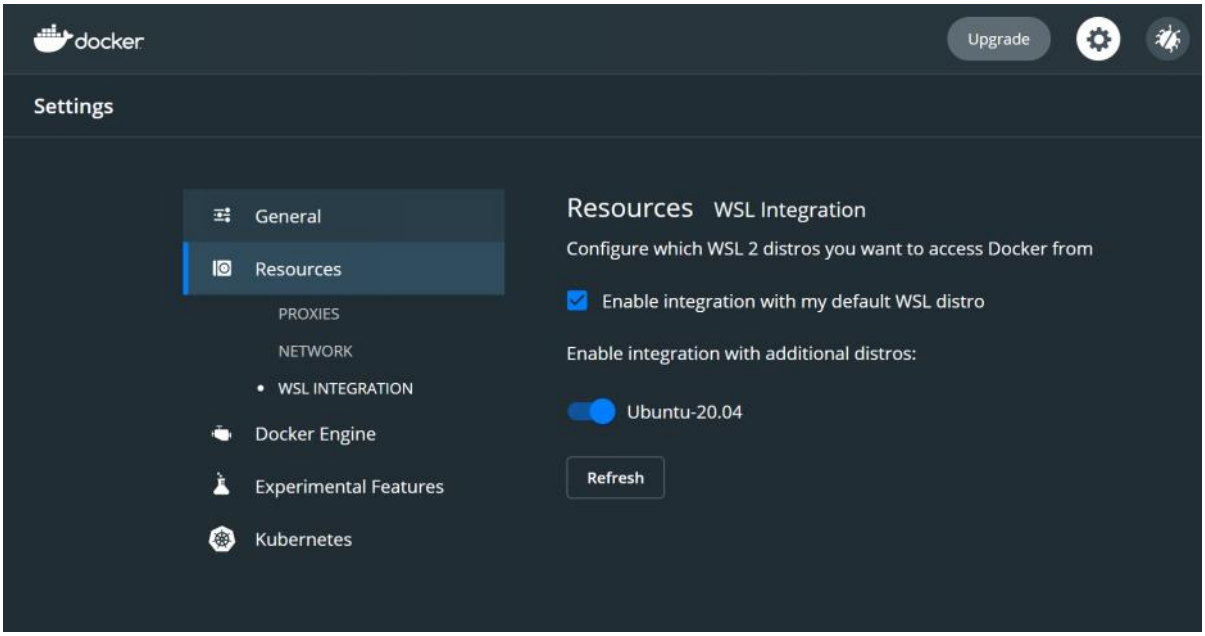
Laravel入門

2021年5月24日　　15:00

2021/05/24
環境構築
https://laravel.com/docs/8.x/installation#getting-started-on-windows

2021-06-10



WSL上でDockerを有効にするには、この画面でディストリビューションごとに有効にする必要がある。ディストリビューション・インストール直後は有効になってないので、Dockerが動いていない時は、この画面を見直してみるべき。

MailHog
http://localhost:8025/ でアクセス可能
通常のWebページは http://localhost

起動
./vendor/bin/sail up
で起動。docker-compose upでも同じことができるようだけど、基本は上でやったほうがいいのかな？

アタッチ
docker-compose execで docker-compose.yml 設定を元にアタッチ可能

webサーバー用コンテナ
名前： laravel.test
PHP 8.0.7 (cli) (built: Jun 4 2021 21:26:10) (NTS)
Laravel Framework 8.46.0

DB用コンテナ
名前：mysql
mysql:8.0というimageからの生成だから、多分MySQLバージョンは8
.envの設定値をもとにmysqlコマンドでDBにアクセスできる

2021-06-11
docker-compose exec はwslでは使えないみたい。コマンドを実行してもレスポンスがなかった。
docker-compose psではコンテナが何も表示されなかった。
Powershellからなら、docker-compose execが実行できた。

2021-06-12
Target class [〇〇〇Controller] does not exist.
そもそも書籍とsailが提供しているバージョンが違う
書籍：6.0

sail：8.46.0

解決方法
https://qiita.com/tamakiiii/items/e71040173fa0a1fcad83 によると、３つ
難しいこと考えずに、名前空間のパスを指定する方法で解決することにした。

改行コードがgit add時にlfからcrlfに変換される
config を core.autocrlf=false に設定されていても、LF-->CRLFになる
.gitattributes で `* text=auto` になっているのが原因っぽい。とりあえず、 `*.php text eol=lf` に設定して先に進むことにした。

シングルアクションコントローラ
https://readouble.com/laravel/8.x/ja/controllers.html のやり方じゃない
と、Laravel 8 は多分だめ

2021-06-13

コーディング規約
https://laravel.com/docs/8.x/contributions#coding-style

2021-06-18

docker-compose upでwarning
```
WARNING: The WWWGROUP variable is not set. Defaulting to a blank string.
WARNING: The WWWUSER variable is not set. Defaulting to a blank string.
```

2021-06-19
docker-compose upでwarning
docker-compose.yml中の変数が外部で定義されているっぽいので、wsl
で ./vendor/bin/sail up したらワーニングは出ないので、docker-compose upは控えた方がいいかも。

2021-06-21
wslから ./vendor/bin/sail up すると、新しいwslコンソールから `docker-compose` が undefindなコマンドになる。ので、docker-compose exec はpowershell からやっていたが、それが使えない時があった。仕方ないから、powershellから `docker-compose up` して、powershellから `docker-compose exec` した。doker-compose.ymlの変数は相変わらず解決できないが、なんかブラウザから普通にアクセスできてる。よく分からない。

2021-06-22
DB
sailの中でMySQLコンテナがすでに入っている。
`docker-compose.yml` にDB接続設定があるが、値は `.env` に書いてある。
`docker-compose exec` でDBコンテナの中に入って、mysqlコマンドで接続することができる。

必要なテーブル
5-1でDBに直接アクセスして作ったテーブルを5-2で使う

```
CREATE TABLE people (
 id int not null primary key auto_increment,
 name varchar(200) not null,
 mail varchar(200),
 age int
);
```

2021-06-29
hasManyでClass not found
モデルクラスを `App/Models` 以下に定義するように本には書かれているが、 同じモデルクラス階層の `board` クラスをreturnさせたいのに、 `App/Board` クラスをreturnするように書かれている。 `App/Models/Board` をreturnするように書き直したら、正常に動作した。
グローバルスコープ定義済みで本が進んでいるのに、where句にid完全一致の `find` メソッドを呼び出すようにかかれていたり、ちょいちょいこの本はトラップが仕掛けられ

ているように思う。

Modelクラス【2021-06-30追記】
本（の中のLaravelバージョン＝6）では、 `artisan` で生成されるModelクラスは、 `namespace App;` になっており、ディレクトリは `app/Models` 以下にあっても、 `App/Modelクラス名` で通る。
自環境（Laravelバージョン8）では、上記と同じディレクトリ構造だが、 `namespace App¥Models;` で生成される。
知ってて注意しないと、バージョン差異によるエラーがなかなか克服するのが難しい。

公式サポート【2021-06-30追記】
出版社のサポートページに触れられていた。出版側も把握済みだったみたい。

viewでModelクラスのカラム・プロパティが呼び出せない
書籍では、最新バージョン違い起因だと思うが、Modelクラスのプロパティとして、DBのカラムの値を使いたい時は、 `protected $fillable` に配列でカラム名を指定しないと使えない。

2021-07-05
npm
laravel.testにアタッチして、
```
node --version
v16.3.0
```
が返ってきた。
https://laravel.com/docs/8.x/sail#executing-node-npm-commands で `sail` を介して実行しているが、直接 `docker-compose up` してる時は、 `docker-compose exec` で各コンテナに直接実行させれば良さそう。

2021-07-08
composer
```
docker-compose exec laravel.test composer --version
Composer version 2.1.3 2021-06-09 16:31:20
```
入っているので、 webサーバーコンテナ上で実行させれば、laravel/uiを使えそう。

2021-07-19

ユニットテスト設定
https://codelikes.com/laravel-tests/#toc13

2021/07/20

テスト実行
以下のコマンドで実行可能。
```
プロジェクトカレントディレクトリ> docker-compose exec laravel.test vendor/bin/phpunit
```

ログ監視
Windows PowerShellで `tail -f` でlaravelのログを監視
```
プロジェクトカレントディレクトリ>Get-Content -Wait -Tail 10  -Path .\storage\logs\laravel.log
```

- 参考
  - https://www.geekfeed.co.jp/geekblog/powershell-Get-Content/
  - https://readouble.com/laravel/8.x/ja/logging.html

テスト用データベース作成
データベースを使ったテストで、データベース接続辺りでエラーになり、HTTPステータスが500になっていた。心当たりはテスト用データベースがないこと。

そこでテスト用データベースを作成することに。開発用データベースはsailで自動的に作ったが、この環境でデータベースを作成することがなかった。そのため、MySQLのデータベース作成から調べることになった。作成コマンドは簡単なので、メモは省略。

MySQLユーザを確認

MySQLにrootで接続して調査。sail環境ではパスワードは、sailユーザと同じ。
sailユーザが使えるテスト用データベースを作るために、確認

```
mysql> select user, host from mysql.user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| root             | %         |
| sail             | %         |
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
```

sailユーザが持ってる権限確認

```
mysql> show grants for 'sail'@'%';
+-------------------------------------------------------+
| Grants for sail@%                                     |
+-------------------------------------------------------+
| GRANT USAGE ON *.* TO `sail`@`%`                      |
| GRANT ALL PRIVILEGES ON `example\_app`.* TO `sail`@`%` |
+-------------------------------------------------------+
```

2021-07-21

テスト用データベース作成

昨日の続き。

sailユーザーは接続ホストはワイルドカードで設定されているため、新しく作ったテスト用データベースの権限を追加すれば良さそう。

sailユーザーに権限付与

先の調査で、
`GRANT ALL PRIVILEGES ON `example_app`.* TO `sail`@`%``
と、あるので、rootユーザーでMySQLに接続し、
`GRANT ALL PRIVILEGES ON test_db.* TO sail;`
を実行して権限付与。

rootを抜けて、sailでMySQLに再接続。データベースを確認。

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| example_app        |
| information_schema |
| test_db            |
+--------------------+
```

- 参考
  - https://www.dbonline.jp/mysql/user/index6.html

Laravel8でのテストの実行

php artisanでもテストが実行できる。というより、そっちの方が本の通りやるより、詳細に情報を出力してくれる。

プロジェクトカレントディレクトリ`> docker-compose exec laravel.test php artisan test`

```
   PASS  Tests\Unit\ExampleTest
  ✓ example

   PASS  Tests\Feature\ExampleTest
  ✓ example
```

```
 FAIL  Tests\Feature\HelloTest
× hello

---

• Tests\Feature\HelloTest > hello
Expected status code 200 but received 500.
Failed asserting that 200 is identical to 500.

at tests/Feature/HelloTest.php:31
   27 ⇂        $response->assertStatus(302);
   28 ⇂
   29 ⇂        $user = User::factory()->make();
   30 ⇂        $response = $this->actingAs($user)->get('/hello');
➜  31 ⇂        $response->assertStatus(200);
   32 ⇂
   33 ⇂        $response = $this->get('/no_route');
   34 ⇂        $response->assertStatus(404);
   35 ⇂    }


Tests:  1 failed, 2 passed
Time:   22.33s
```

実際はカラーハイライトもされてて、もっとわかりやすい。

HTTPステータス500でテストが失敗する

Laravelログを見ると、

```
#11 /var/www/html/app/Http/Controllers/HelloController.php(22): Illuminate¥¥Database¥¥Eloquent
¥¥Builder->simplePaginate()
```

当該箇所のコードは

```
$items = Person::orderBy($sort, 'asc')
    ->simplePaginate(5);
```

ページネーション用データがないのが直接原因だと思う。

これは、明日調査、修正する。

それと、sortパラメータ必須のコードになっているので、当該箇所を

```
$sort = isset($request->sort) ? $request->sort : 'id';
$items = Person::orderBy($sort, 'asc')
    ->simplePaginate(5);
```

に書き換え。

2021-07-22

Laravel8.xでのユニットテスト用データ作成方法

本の「ダミーレコードの用意」の所。

Laravel6.x

本の環境Laravel6.xでは、database/factories/UserFactory.php は、

```
use Faker\Generator as Faker;
use Illuminate\Support\Str;

$factory->define(App\User::class, function (Faker $faker) {
    return [
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'email_verified_at' => now(),
        'password' => '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', //
password
        'remember_token' => Str::random(10),
    ];
});
```

貼り付け元 <https://readouble.com/laravel/6.x/ja/database-testing.html>

で、クロージャを使った形で説明されている。

peopleデータ追加に関しては、UserFactoryに追記することで対応している。

ファクトリの使用方法は、

```
$user = factory(App\User::class)->make();
```

貼り付け元 <https://readouble.com/laravel/6.x/ja/database-testing.html>
をmakeではなく、createを使っている。
この方法は、8.xでは使えない。

Laravel8.x

`database/factories/UserFactory.php` は、

```php
<?php
namespace Database¥Factories;
use App¥Models¥User;
use Illuminate¥Database¥Eloquent¥Factories¥Factory;
use Illuminate¥Support¥Str;
class UserFactory extends Factory
{
    /**
     * The name of the factory's corresponding model.
     *
     * @var string
     */
    protected $model = User::class;
    /**
     * Define the model's default state.
     *
     * @return array
     */
    public function definition()
    {
        return [
            'name' => $this->faker->name(),
            'email' => $this->faker->unique()->safeEmail(),
            'email_verified_at' => now(),
            'password' => '$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', // password
            'remember_token' => Str::random(10),
        ];
    }
    /**
     * Indicate that the model's email address should be unverified.
     *
     * @return ¥Illuminate¥Database¥Eloquent¥Factories¥Factory
     */
    public function unverified()
    {
        return $this->state(function (array $attributes) {
            return [
                'email_verified_at' => null,
            ];
        });
    }
}
```

と、クロージャではなく、クラス化されている模様。
これに倣って、PersonFactoryクラスを作り、テストクラスで呼び出す。

PersonFactoryクラス

```php
<?php
namespace Database¥Factories;
use App¥Models¥Person;
use Illuminate¥Database¥Eloquent¥Factories¥Factory;
class PersonFactory extends Factory
{
    /**
     * The name of the factory's corresponding model.
     *
     * @var string
     */
    protected $model = Person::class;
    /**
     * Define the model's default state.
     *
     * @return array
     */
    public function definition()
    {
        return [
            'name' => $this->faker->name(),
            'mail' => $this->faker->unique()->safeEmail(),
            'age' => random_int(1, 99),
        ];
    }
}
```

ファクトリ使用

tests\Feature\HelloTest.php の使用箇所抜粋。

```
$user = User::factory()->make();
Person::factory()->make();
$response = $this->actingAs($user)->get('/hello');
$response->assertStatus(200);
```

peopleデータを使った検証はないため、作成したデータは変数格納をしなかった。

結果

プロジェクト・カレント・ディレクトリ> docker-compose exec laravel.test php artisan test

```
   PASS  Tests\Unit\ExampleTest
  ✓ example

   PASS  Tests\Feature\ExampleTest
  ✓ example

   PASS  Tests\Feature\HelloTest
  ✓ hello

  Tests:  3 passed
  Time:   10.07s
```

問題なくクリアできた。

モデルの生成とデータの保存

User::factory()->make(); が、モデルのインスタンスを生成するだけなのに対し、
本で取り扱っている User::factory()->create(); は、モデルのインスタンス化とデータの保存を行う。

- 参考
  - https://readouble.com/laravel/8.x/ja/database-testing.html

テストデータをcreateで永続化したら、テストデータベースにデータが書き込まれない

設定が何か間違っている。
https://www.webopixel.net/php/1430.html を見ながら、設定を書き直し中。

データベースCLIへの接続

以下のコマンド実行で、docker-compose execでmysqlコンテナに手動でアタッチしなくても、一気にmysqlコンテナアタッチとMySQLへのログインをパスワード入力もなしで実行できる。
プロジェクト・カレント・ディレクトリ> docker-compose exec laravel.test php artisan db <接続設定名>

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 52
Server version: 8.0.25 MySQL Community Server - GPL
～（省略）～
mysql>
```

すごく楽ちん。もっと早く知りたかった。

git logでcommitしたファイル名も表示する

```
$ git log --stat
commit ae69a46fe3a8136fe0d07411edb34c45ac40acd8 (HEAD -> 7-5, origin/7-5)
Author: t-sagiri <sagirix.t@gmail.com>
Date:   Thu Jul 22 10:17:55 2021 +0900

    ユーザーデータを使った認証ページへのアクセステスト作成

 database/factories/PersonFactory.php | 31 ++++++++++++++++++++++++++++++
 tests/Feature/HelloTest.php          | 19 +++++++++++++------
 2 files changed, 44 insertions(+), 6 deletions(-)
```

- 参考
  - https://mimirswell.ggnet.co.jp/blog-211

2021-07-23

git　ファイルのロールバック

ずっと前にcommitした phpunit.xml をcommit前に戻したいので、調べた。

1. 変更したcommitの１つ前のコミットidを控えておく
2. git checkout [1で控えたコミットid] [ファイルパス]

- 参考
  - https://qiita.com/rch1223/items/9377446c3d010d91399b

テストデータをcreateで永続化したら、テストデータベースにデータが書き込まれない

https://www.webopixel.net/php/1430.html の通り、ファイルを編集

```
config\database.php
    'connections' => [
        'sqlite' => [
～（略）～
        ],
        'testing' => [
            'driver' => 'mysql',
            'database' => env('DB_DATABASE', 'forge'),
            'prefix' => '',
        ],
        'pgsql' => [
～（略）～
```

.env.testing

```
DB_CONNECTION=testing
```

phpunit.xmlのAPP_ENVがtestingになっていることだけ確認。

結果

```
 PASS  Tests\Unit\ExampleTest
  ✓ example

  FAIL  Tests\Feature\ExampleTest
  × example

  FAIL  Tests\Feature\HelloTest
  × hello

  ---

  • Tests\Feature\ExampleTest > example
   Illuminate\Encryption\MissingAppKeyException

  No application encryption key has been specified.

  at vendor/laravel/framework/src/Illuminate/Encryption/EncryptionServiceProvider.php:79
     75 ▕     protected function key(array $config)
     76 ▕     {
     77 ▕         return tap($config['key'], function ($key) {
     78 ▕             if (empty($key)) {
  ➜  79 ▕                 throw new MissingAppKeyException;
     80 ▕             }
     81 ▕         });
     82 ▕     }
     83 ▕ }

      +18 vendor frames
  19  tests/Feature/ExampleTest.php:17
      Illuminate\Foundation\Testing\TestCase::get()

  • Tests\Feature\HelloTest > hello
   Illuminate\Encryption\MissingAppKeyException

  No application encryption key has been specified.

  at vendor/laravel/framework/src/Illuminate/Encryption/EncryptionServiceProvider.php:79
     75 ▕     protected function key(array $config)
     76 ▕     {
     77 ▕         return tap($config['key'], function ($key) {
     78 ▕             if (empty($key)) {
  ➜  79 ▕                 throw new MissingAppKeyException;
     80 ▕             }
     81 ▕         });
     82 ▕     }
     83 ▕ }
```

```
      +18 vendor frames
  19  tests/Feature/HelloTest.php:24
      Illuminate\Foundation\Testing\TestCase::get()


  Tests:  2 failed, 1 passed
  Time:   8.66s
```

別サイトを参照して、ファイルを編集し直す。

.env.testing

```
APP_ENV=test_db
APP_KEY=
DB_CONNECTION=testing
```

キャッシュクリア

.env系ファイル編集後は必須らしいので、以下コマンド実行でキャッシュクリア。

```
> docker-compose exec laravel.test php artisan config:clear
Configuration cache cleared!
```

新しいAPP_KEYを発行

```
> docker-compose exec laravel.test php artisan key:generate --env=testing
Application key set successfully.
```

migrate

```
> docker-compose exec laravel.test php artisan migrate --env=testing

In Connection.php line 692:

  Undefined variable $host (SQL: select * from information_schema.tables where table_schema = test
  _db and table_name = migrations and table_type = 'BASE TABLE')


In MySqlConnector.php line 154:

  Undefined variable $host
```

ホスト等がわからないらしい。参照サイトでは、mysql接続設定をコピペしていたので、コピペして、再度実行。

今度は、

```
In Connection.php line 692:

  SQLSTATE[HY000] [2002] Connection refused (SQL: select * from information_schema.tables where ta
  ble_schema = forge and table_name = migrations and table_type = 'BASE TABLE')


In Connector.php line 70:

  SQLSTATE[HY000] [2002] Connection refused
```

https://qiita.com/isaatsu0131/items/50f8dca389b60a1fd5b5 の通り、.env系の接続設定が間違っていると思う。

https://codelikes.com/laravel-tests/#toc13 を参考に、.env.testingを

APP_ENV=test_db

APP_KEY=base64:MdOSs4vwBwhRt0UAekLa/mLSoNKFgKdSZiOoeTjmyAk=

DB_HOST=mysql

DB_PORT=3306

DB_DATABASE=test_db

DB_CONNECTION=testing

DB_USERNAME=sail

DB_PASSWORD=password

phpunit.xmlを

```
<php>
    <server name="APP_ENV" value="testing"/>
    <server name="BCRYPT_ROUNDS" value="4"/>
    <server name="CACHE_DRIVER" value="array"/>
    <!-- <server name="DB_CONNECTION" value="sqlite"/> -->
    <!-- <server name="DB_DATABASE" value=":memory:"/> -->
    <server name="MAIL_MAILER" value="array"/>
    <server name="QUEUE_CONNECTION" value="sync"/>
    <server name="SESSION_DRIVER" value="array"/>
    <server name="DB_CONNECTION" value="mysql"/>
    <!-- <server name="TELESCOPE_ENABLED" value="false"/> -->
```

に編集。

configのキャッシュをクリアして、migrateし直すと、

Migration table created successfully.

Migrating: 2014_10_12_000000_create_users_table

Migrated: 2014_10_12_000000_create_users_table (262.92ms)

Migrating: 2014_10_12_100000_create_password_resets_table

Migrated: 2014_10_12_100000_create_password_resets_table (148.37ms)
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (194.23ms)
Migrating: 2021_06_24_013828_create_people_table
Migrated: 2021_06_24_013828_create_people_table (69.08ms)
Migrating: 2021_06_28_080043_create_boards_table
Migrated: 2021_06_28_080043_create_boards_table (58.37ms)
Migrating: 2021_06_30_004335_create_restdata_table
Migrated: 2021_06_30_004335_create_restdata_table (57.23ms)
Migrating: 2021_07_01_082510_create_sessions_table
Migrated: 2021_07_01_082510_create_sessions_table (250.44ms)

テスト用DBを確認

```
mysql> show tables;
+-------------------+
| Tables_in_test_db |
+-------------------+
| boards            |
| failed_jobs       |
| migrations        |
| password_resets   |
| people            |
| restdata          |
| sessions          |
| users             |
+-------------------+
```

今度は正しいデータベースにテーブルが作成された。

テスト結果も、成功。

```
> docker-compose exec laravel.test php artisan test

   PASS  Tests\Unit\ExampleTest
  ✓ example

   PASS  Tests\Feature\ExampleTest
  ✓ example

   PASS  Tests\Feature\HelloTest
  ✓ hello

  Tests:  3 passed
  Time:   8.86s
```

念の為、テスト用DBのデータを参照。

```
mysql> select * from users;
+----+---------------+-------------------------+---------------------+--------------------------------------------------------------+------
----------+---------------------+---------------------+
| id | name          | email                   | email_verified_at   | password
                                | remember_token | created_at          | updated_at          |
+----+---------------+-------------------------+---------------------+--------------------------------------------------------------+------
----------+---------------------+---------------------+
|  1 | Dovie Stracke | mayer.elijah@example.com | 2021-07-23 02:04:44 | $2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi |
ewxYd7Q298     | 2021-07-23 02:04:44 | 2021-07-23 02:04:44 |
+----+---------------+-------------------------+---------------------+--------------------------------------------------------------+------
----------+---------------------+---------------------+
1 row in set (0.00 sec)
```

永続化データも正しく生成されたことを確認できた。
ふー、やれやれ。時間かかった(；´Д｀)